

PDF Script 3, peer to peer: Jordi

- NIVEL 1 EJERCICIO 1: Tu tarea es diseñar y crear una mesa llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito.
- La nueva tabla debe ser capaz de identificar de manera única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company").
- Después de crear la mesa será necesario que ingreses la información del documento denominado "dades_introducir_credit".
- Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

a- creamos la tabla llamada credit_card según info de 'datos_introducir_sprint3_credtit.sql', el id será la PK, será el campo que nos conectará a la tabla 'transaction'

```
USE transactions;
CREATE TABLE IF NOT EXISTS user (
  id CHAR(10) PRIMARY KEY,
  name VARCHAR(100),
  surname VARCHAR(100),
  phone VARCHAR(150),
  email VARCHAR(150),
  birth_date VARCHAR(100),
  country VARCHAR(150),
  city VARCHAR(150),
  postal_code VARCHAR(100),
  address VARCHAR(255) );
SELECT * FROM user; -- 5000 rows
```

Output		
Time	Action	Message
7 20:58:50	USE transactions	0 row(s) affected
		Duration / Fetch 0.000 sec

```
:6
:7
:8 • CREATE TABLE IF NOT EXISTS credit_card(
:9   id VARCHAR(20) PRIMARY KEY,
:10  iban VARCHAR(50) NOT NULL,
:11  pan VARCHAR(25) NOT NULL,
:12  pin VARCHAR(4) NOT NULL,
:13  cvv VARCHAR(4) NOT NULL,
:14  expiring_date VARCHAR(20) NOT NULL
:15 );
:16 • SHOW tables;
:17 • SELECT * FROM credit_card;
:18
:19
```

- comprobamos que las tablas existen: (4 tablas): 'transaction', 'company', 'user', 'credit_card'
- " " ha cargado los datos: 5000 rows
- La nueva tabla debe ser capaz de identificar de manera única cada tarjeta y
- establecer una relación adecuada con las otras dos tablas ("transaction" y "company").
- Vamos a relacionar la tabla 'credit_card' con la tabla 'transaction' a través de 'credit_card_id':
- Vamos a relacionar la tabla 'transaction' con la tabla 'user' a través de 'user_id', antes necesitamos que tengan mismo tipo de dato

- ALTER table transaction

```
ADD CONSTRAINT fk_transaction_credit_card_id
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

```
-- 100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0 3.859 sec
```

- ALTER TABLE user

```
MODIFY id INT;
```

- ALTER table transaction

```
ADD CONSTRAINT fk_transaction_user
FOREIGN KEY (user_id)
REFERENCES user(id);
```

```
-- 100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0 5.312 sec
```

-- EJERCICIO 2

-- El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a la tarjeta de crédito

-- con ID CcU-2938. La información que debe mostrarse para este registro es:

TR323456312213576817699999.

-- Recuerda mostrar que el cambio se realizó.

```
92 -- antes del cambio
93 • SELECT * FROM credit_card
94 WHERE id = 'CcU-2938'; -- 'CcU-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22'
95
96 • UPDATE credit_card
97 SET iban = 'TR323456312213576817699999'
98 WHERE id = 'CcU-2938'; -- 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 0.000 sec
99
100 -- después de actualizar:
101 • SELECT * FROM credit_card
102 WHERE id = 'CcU-2938'; -- 'CcU-2938', 'TR323456312213576817699999', '5424465566813633', '3257', '984', '10/30/22'
```

result Grid						
id	iban	pin	cvv	expiring_date	fecha_actual	
CcU-2938	TR323456312213576817699999	3257	984	10/30/22	2026-04-07	
NULL	NULL	NULL	NULL	NULL	NULL	

#	Time	Action	Message	Duration /
1	21:10:20	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec /

```
-- EJERCICIO 3:
-- En la tabla "transaction" ingresa una nueva transacción con la siguiente información:
-- Id      108B1D1D-5B23-A76C-55EF-C568E49A99DD
-- credit_card_id      CcU-9999
-- company_id b-9999
-- user_id      9999
-- lat      829.999
-- longitude      -117.999
-- amount      111.11
-- declined      0
-- SELECT * FROM transaction; -- antes de ingresar una nueva transacción: 100000 rows

-- INSERT INTO transaction (Id, credit_card_id, company_id, user_id, lat, longitude, amount,
declined)
-- VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999,
829.999, -117.999, 111.11, 0);
-- da error porque
-- SELECT * FROM credit_card WHERE id = 'CcU-9999'; -- 0 rows NO EXISTE
credit_card_id = 'CcU-9999'
-- SELECT * FROM company WHERE id = 'b-9999'; -- " " " " company_id =
'b-9999'

-- entonces tenemos que crear los valores que faltan:
```

The screenshot shows a database management tool interface with a SQL editor and a results grid. The SQL editor contains the following queries:

```
134
135
136 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date)
137 VALUES ('CcU-9999', 'E5000000000000000000', '0000000000000000', '0000', '000', '00/00/00'); -- 1 row
138
139 • INSERT INTO company (id, company_name, phone, email, country, website)
140 VALUES ('b-9999', 'company_test', '000000000', 'test@test.com', 'country_test', 'www.test.com'); -- 1 row
141 -- tambien el la tabla user porque está relacionada:
142 • INSERT INTO user (id) VALUES ('9999'); -- 1 row
143 -- probamos ahora:
144 • INSERT INTO transaction (Id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
145 VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0); -- 1 row
146 • SELECT * FROM transaction; -- después de ingresar la nueva transacción: 100001 rows
147 • SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
148 -- devuelve '108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', NULL, '111.11', '0'
```

The results grid shows the following data:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0

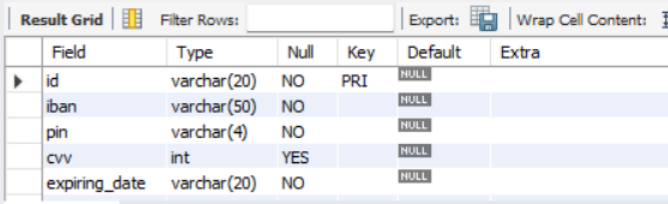
The output section shows the following message:

```
1 21:10:20 SELECT * FROM credit_card WHERE id = 'CcU-2938'
Message
1 row(s) returned
Duration / I
0.000 sec /
```

-- EJERCICIO 4

- Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card.
- Recuerda mostrar el cambio realizado.

```
157
158 • SHOW columns FROM credit_card;    -- antes de hacer el cambio: 6 rows
159 • ALTER TABLE credit_card
160     DROP COLUMN pan;
161 • SHOW columns FROM credit_card;    -- después de hacer el cambio: 5 rows
162
```



Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PRI	NULL	
iban	varchar(50)	NO		NULL	
pin	varchar(4)	NO		NULL	
cvv	int	YES		NULL	
expiring_date	varchar(20)	NO		NULL	

Result 50 x

Output

Action Output

#	Time	Action	Message
2	21:13:41	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD'	1 row(s) returned

-- NIVEL 2, EJERCICIO 1

- Elimina de la tabla transaction el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

```
170
171 • SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'; -- primero verificamos 1 row
172 • DELETE FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
173 • SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'; -- después comprobamos 0 rows
174
175
176
177
```



id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 51 x

Output

Action Output

#	Time	Action	Message
3	21:16:41	SHOW columns FROM credit_card	6 row(s) returned

- EJERCICIO 2: La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas.

- Se ha solicitado crear una VISTA que proporcione detalles clave sobre las compañías y sus transacciones.
- Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información:
- Nombre de la compañía, Teléfono de contacto. País de residencia. Promedio de compra realizado por cada compañía.
- Presenta la vista creada, ordenando los datos de mayor a menor media de compra.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'transactions' database is selected, and the 'Schemas' tab is active. The 'Schema: transactions' is displayed. The main pane shows the SQL script for creating the 'VistaMarketing' view. The script is as follows:

```

190 • CREATE VIEW VistaMarketing AS
191 SELECT company_name, phone, country, ROUND(AVG(amount),2) AS promedio_compra
192 FROM company JOIN transaction
193 ON company.id = transaction.company_id
194 GROUP BY company_name, phone, country
195 ORDER BY promedio_compra DESC;
196
197 • SELECT * FROM VistaMarketing; -- 101 rows
198
199

```

Below the script, the 'Result Grid' shows the data returned by the second query. The data is as follows:

company_name	phone	country	promedio_compra
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Pretium Neque Corp.	07 77 48 55 28	Australia	276.158330
Urna Convallis Associates	06 01 24 77 04	United States	274.235011
At Associates	09 56 61 10 65	New Zealand	272.214870
Metus Vitae Associates	08 25 44 40 66	Australia	270.080965

The 'Output' pane shows the execution of the query, with the message '0 row(s) returned'.

-- EJERCICIO 3

- Filtra la vista VistaMarketing para mostrar sólo las compañías que tienen su país de residencia en "Germany"

The screenshot shows the SQL Server Enterprise Manager interface. The main pane shows the SQL script for filtering the 'VistaMarketing' view. The script is as follows:

```

209 • SELECT * FROM VistaMarketing
210 WHERE country = "Germany"; -- 8 rows

```

Below the script, the 'Result Grid' shows the data returned by the query. The data is as follows:

company_name	phone	country	promedio_compra
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Nunc Interdum Incorporated	05 18 15 48 13	Germany	259.319156
Convallis In Incorporated	06 66 57 29 50	Germany	257.745376
Ac Industries	09 34 65 40 60	Germany	255.147288
Rutrum Non Inc.	02 66 31 61 09	Germany	255.136927

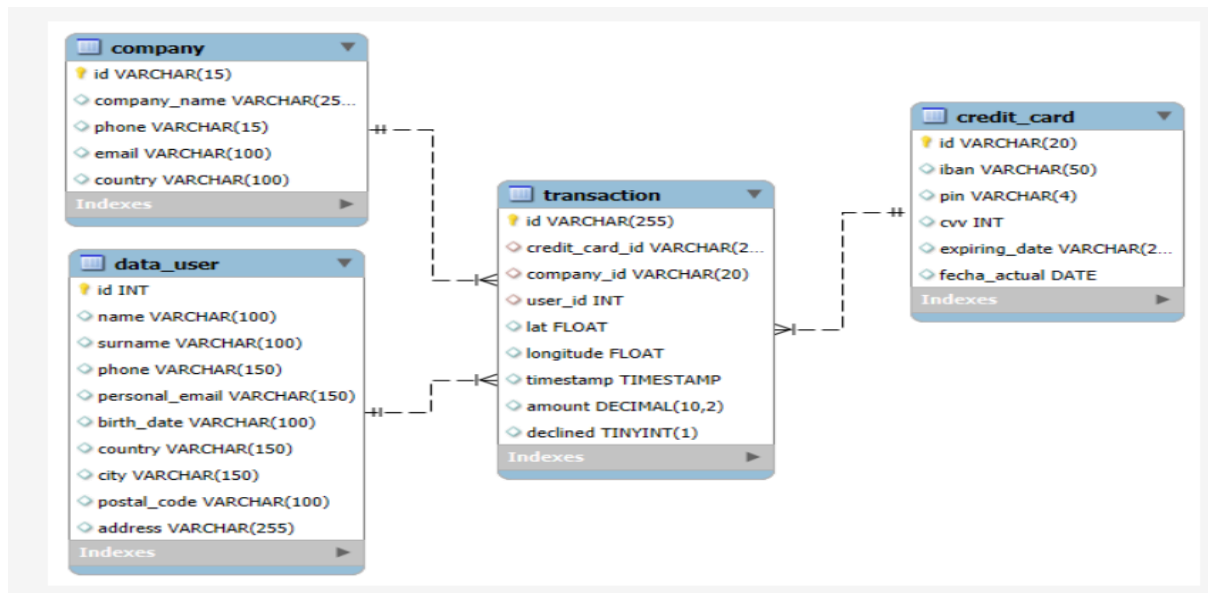
The 'Output' pane shows the execution of the query, with the message '8 row(s) returned'.

-- NIVEL 3, EJERCICIO 1

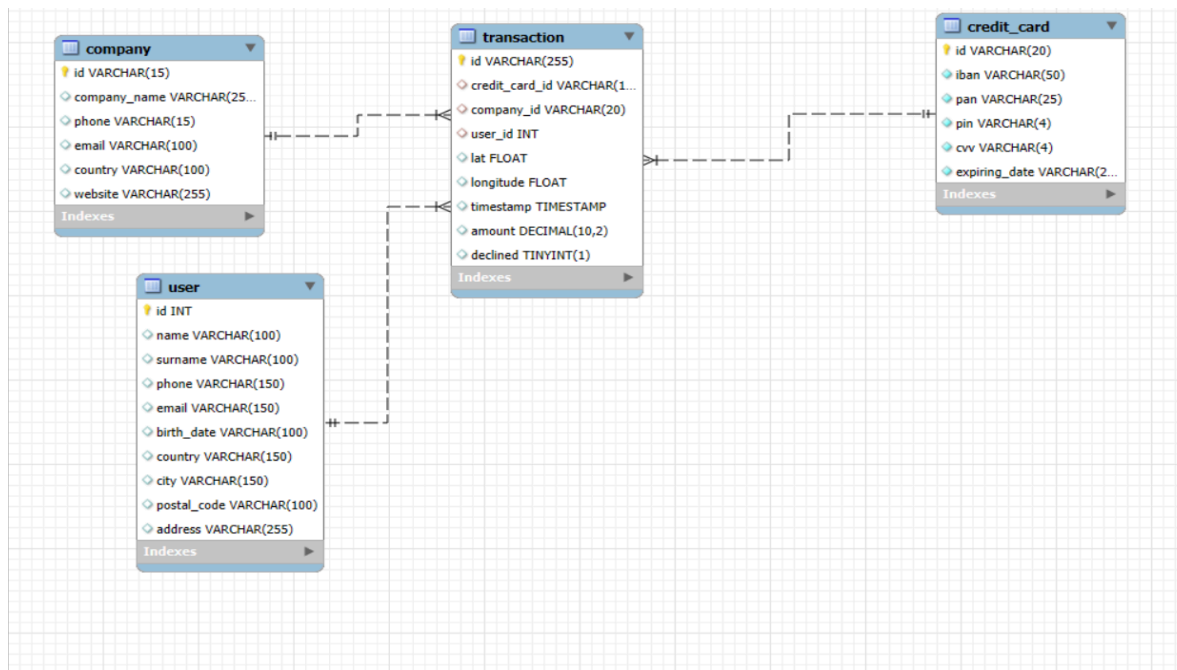
-- La semana próxima tendrás una nueva reunión con los gerentes de marketing.

-- Un compañero de tu equipo realizó modificaciones en la base de datos, pero no recuerda cómo las realizó.

-- Te pide que le ayudes a dejar los comandos ejecutados para obtener el siguiente diagrama:



Este es el EER antes de cambios:



-- tenemos en primer lugar un EER modificado, y más abajo el EER actual. Necesitamos hacer los siguientes cambios:

-- 3.1.1 tabla company: eliminar columna website:

227

```
228 • ALTER TABLE company
229     DROP COLUMN website;
230 • SHOW COLUMNS FROM company;    -- 5 rows, antes 6
231
```

-- 3.1.2 tabla transaction cambiar credit_card_id VARCHAR (15) a credit_card_id VARCHAR (20)

```
235 • ALTER TABLE transaction
236     MODIFY credit_card_id VARCHAR (20);
237
```

-- 3.1.3 tabla credit_card añadir fecha_actual DATE

Table: credit_card Columns: <table border="0"><tr><td>id</td><td>varchar(20) PK</td></tr><tr><td>iban</td><td>varchar(50)</td></tr><tr><td>pin</td><td>varchar(4)</td></tr><tr><td>cvv</td><td>int</td></tr><tr><td>expiring_date</td><td>varchar(20)</td></tr><tr><td>fecha_actual</td><td>date</td></tr></table>	id	varchar(20) PK	iban	varchar(50)	pin	varchar(4)	cvv	int	expiring_date	varchar(20)	fecha_actual	date	<pre>244 245 • ALTER TABLE credit_card 246 ADD fecha_actual DATE DEFAULT (CURDATE()); -- 5001 rows, 6 columnas , antes 5 247 248</pre>
id	varchar(20) PK												
iban	varchar(50)												
pin	varchar(4)												
cvv	int												
expiring_date	varchar(20)												
fecha_actual	date												

-- 3.1.4 tabla user cambiar nombre de la tabla por data_user

Information :::::::::::::::::::::::::::::::::: Table: data_user Columns:	<pre>252 253 • RENAME TABLE user 254 TO data_user; 255</pre>
--	--

-- 3.1.5 tabla credit_card, cambiar 'cvv VARCHAR(4)' por 'cvv INT'

Table: data_user Columns:	<pre>261 • ALTER TABLE data_user 262 RENAME COLUMN email TO personal_email; 263</pre>
---	---

-- 3.1.6 tabla data_user renombrar columna 'email' a 'personal_email';

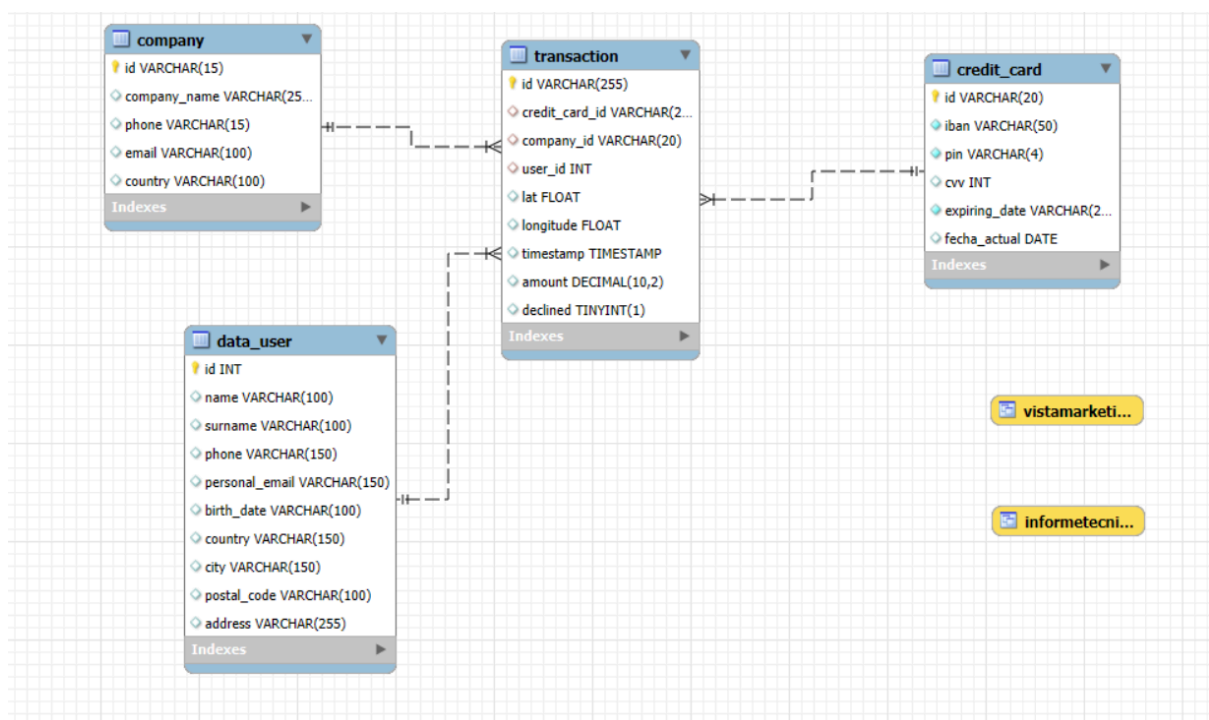
Table: data_user

Columns:

```
264 • ALTER TABLE data_user
265 RENAME COLUMN email TO personal_email;
266
```

-- DESPUÉS DE LOS CAMBIOS LOS DOS EER COINCIDEN

Este es el EER después de los cambios:



-- NIVEL 3: EJERCICIO 2

-- La empresa también os pide crear una VISTA llamada "InformeTecnico" que contenga la siguiente información:

-- ID de la transacción

-- Nombre del usuario/a

-- Apellido del usuario/a

-- IBAN de la tarjeta de crédito usada.

-- Nombre de la compañía de la transacción realizada.

-- Asegúrese de incluir información relevante de las tablas que conoceréis y utilice alias para cambiar de nombre columnas según sea necesario.

-- Muestra los resultados de la vista, ordena los resultados de forma descendente en función de la variable ID de transacción.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'tienda_online' database with a 'transactions' schema containing tables 'company', 'credit_card', 'data_user', and 'transaction', and a view 'informetecnico'. The 'Columns' pane for 'informetecnico' lists: Identificador (varchar(255)), Nombre (varchar(100)), Apellido (varchar(100)), iban (varchar(50)), Empresa (varchar(255)), and País (varchar(150)).

The main pane shows the SQL script for creating and querying the view:

```
283
284 • CREATE VIEW InformeTecnico AS
285 SELECT t.id AS Identificador, d.name AS Nombre, d.surname AS Apellido, cc.iban, c.company_name AS Empresa, d.country AS País
286 FROM transaction t
287 JOIN data_user d
288 ON t.user_id = d.id
289 JOIN credit_card cc
290 ON t.credit_card_id = cc.id
291 JOIN company c
292 ON t.company_id = c.id
293 ORDER BY t.id DESC;
294
295 • SELECT *
296 FROM InformeTecnicoinformetecnico; -- 100.000 row(s) returned
297
```

The 'Result Grid' shows the first five rows of the view's output:

Identificador	Nombre	Apellido	iban	Empresa	País
FFFD31D6-9495-47CE-B54A-7D88E1CC274B	Bmrgli	Tprvmrc	XX794814451211289182490922	Turpis Company	United Kingdom
FFFCF76D-ECF0-4985-A2D0-82A7B75998FC	Dffied	Vilqjdi	XX636251701647892036676034	Amet Nulla Donec Corporation	Netherlands
FFFC9E8D-27C7-44DE-98F2-7533EF4DF126	Securp	Faoivqfy	XX162677143304223631437567	Nunc Interdum Incorporated	Sweden
FFFB270D-F53A-4D3D-9666-E5307C53CC84	Ggziqa	Uirzqjh	XX395114267082019952567052	Viverra Donec Foundation	Portugal
FFFB3CE-234E-408C-A8EF-F9CAD577224A	Yshimq	Zpgsleed	XX8845462156537570367941	Convalis In Incorporated	Germany

The 'Output' pane shows the execution of the query: 'SELECT * FROM InformeTecnico' completed at 21:52:21, returning 100,000 rows in 2.031 seconds.

Fin del script 3